# Use cases for polar data discovery with the POLARIN Data Hub

**Audience level: Beginner / Intermediate / Advanced (depending on the use case)**

**Estimated reading time: 30 minutes**

**Contributor: Rachele Bordoni (ETT)**

**Date created: 27-05-2026**

**Date last modified: 09-06-2026**



The POLARIN Data Hub brings together polar research data from across a wide network of Arctic and Antarctic Research Infrastructures. But knowing that the data exists is only the first step — users also need to know *how* to find what they need, *how* to explore it, and *how* to get it into their analysis workflow. This notebook presents three representative use cases drawn from real POLARIN user personas, each showing a complete workflow from data discovery to download or analysis. By the end of this notebook, you will have seen how three different types of users — an oceanographer, a PhD student, and a policy analyst — can each use the POLARIN Data Hub to meet their specific data needs.

---

## Learning objectives

By the end of this notebook, you will:

1. Know how to search and filter the Data Selection by Earth System, Location, and Layer type to identify ERDDAP-accessible datasets.
2. Be able to retrieve and visualize oceanographic data programmatically via ERDDAP.
3. Understand how to assess data availability and identify gaps using the Data Hub tools.
4. Know how to explore polar data interactively using the Data Viewer and export results.

## Prerequisites

- Basic familiarity with Python and Jupyter Notebooks (for Use Case 1).
- A web browser and internet access.
- (Optional but recommended) Completion of *Notebook 5: How to use the POLARIN Data Hub?*

### User Personas

The three use cases in this notebook correspond to three user profiles that represent the POLARIN community:

| Use Case | Persona | Main tools used |
|---|---|---|
| 1 | Oceanographer / biogeochemist | Data Selection, ERDDAP (Python), Data Viewer |
| 2 | PhD student / graduate researcher | Data Hub landing page, Data Selection |
| 3 | Policy analyst / decision maker | Data Selection, Data Viewer |

---

# Use Case 1: Advanced analysis of water column properties

**Persona**: Oceanographer or biogeochemist — an advanced user with scripting experience.

**Goal**: Retrieve and analyze multi-depth oceanographic data (temperature, salinity, dissolved oxygen) from a specific polar region to understand water column gradients and their seasonal variability.

**Workflow overview**:

1. Use the Data Selection to identify datasets with ERDDAP access, filtered by Earth System and Location.
2. Explore the dataset metadata on GeoNetwork by clicking on the GeoNetwork button and search for the ERDDAP link in the metadata record.
3. Note the ERDDAP dataset ID of the datasets of interest.
4. Inspect available variables on ERDDAP and identify datasets containing the parameters of interest.
5. Query the data programmatically via ERDDAP using Python.
6. Visualize water column profiles and time series.
7. Export the data for further analysis.

## Step 1 — Filter the Data Selection to find ERDDAP-accessible datasets

Navigate to the POLARIN Data Selection and apply the following filters:

- **Location**: select **Arctic** (or Antarctic, depending on your study area).
- **Earth System**: select **Ocean** to focus on oceanographic datasets.
- **Layer type**: select **Layers with data** — this is the key filter that restricts results to datasets that have actual data accessible via ERDDAP (shown as  green markers in the Data Viewer), excluding metadata-only entries.

Each dataset has a colored markers in the entry. There are two types of markers — and the distinction matters:

| Marker color | What it means |
|---|---|
|  **Green** | Data layer/Layers with data — the dataset contains plottable data. Their values can be displayed and plotted in the Data Viewer |

|  **Blue** | Metadata only — the dataset has a metadata record in the catalogue, but no plottable data is directly accessible through the Viewer.

Browse the filtered results. Each entry shows the dataset title, description, Earth System and distributing RI. Search for the entries that look relevant to your research question and search for the ERDDAP dataset ID in the GeoNetwork metadata of that entry - you will use these IDs in the next steps to query the data via ERDDAP.

**dataset ID example** ERDDAP dataset link: https://erddap.s4polarin.eu/erddap/tabledap/cnr_iadc_f5ff_4134_70dd.html
dataset ID: cnr_iadc_f5ff_4134_70dd

> At this stage the Data Selection does not filter by specific variable (e.g., temperature, salinity) — that level of filtering happens in the next step directly on ERDDAP, where you can inspect exactly which variables each dataset contains.

## Step 2 — Inspect variables on ERDDAP

Now that you have a list of candidate dataset IDs from the Data Selection, use ERDDAP to check which variables each dataset contains. This is how you identify which datasets have the specific parameters you need (e.g., temperature, salinity, dissolved oxygen).

You can do this in two ways:

**Option A — via the ERDDAP web interface (no code needed)**:

1. Go to https://erddap.s4polarin.eu/erddap/tabledap/index.html
2. Find your dataset ID in the list and click on it.
3. The dataset page lists all available variables with their descriptions and units.

**Option B — programmatically (see Step 3 below)**: Use the ERDDAP metadata API to inspect variables for all candidate datasets at once.

Once you have identified which datasets contain the variables of interest, note their **ERDDAP dataset ID** — you will use it to query the actual data in the steps that follow.

In [ ]:
```python
# Install required packages if needed
# !pip install requests pandas matplotlib

import requests
import pandas as pd
import matplotlib.pyplot as plt
import io
import warnings

warnings.filterwarnings("ignore")

ERDDAP_BASE_URL = "https://erddap.s4polarin.eu/erddap"
```

## Step 3 — Inspect the dataset

Before querying data, it is good practice to inspect the dataset's available variables and station identifiers. Here we use a CTD dataset from the POLARIN ERDDAP as an example — replace `DATASET_ID` with the ID of the dataset you identified in Step 1.

In [ ]:
```python
# Replace with the dataset ID you identified in the Data Selection
DATASET_ID = "cnr_iadc_f5ff_4134_70dd"
dataset_url = f"{ERDDAP_BASE_URL}/tabledap/{DATASET_ID}"

# --- List available variables ---
metadata_url = f"{ERDDAP_BASE_URL}/info/{DATASET_ID}/index.csv"
try:
    metadata_resp = requests.get(metadata_url, timeout=30)
    metadata_resp.raise_for_status()
    metadata_df = pd.read_csv(io.StringIO(metadata_resp.text), sep=',')
    variables_df = metadata_df.loc[metadata_df['Row Type'].isin(['variable',
    variables_df = variables_df[['Variable Name', 'Data Type']].reset_index(
    print(f"Dataset: {DATASET_ID}")
    print("Available variables:")
    print(variables_df.to_string(index=False))
except Exception as e:
    print(f"Could not retrieve metadata for {DATASET_ID}: {e}")
```

In [ ]:
```
# --- List unique station IDs ---
# Note: not all datasets have a 'station_id' column.
# If the query fails, check the variable list above for the correct identifi
try:
    platforms_resp = requests.get(
        dataset_url + ".csv?station_id&distinct()",
        timeout=30
    )
    platforms_resp.raise_for_status()
    # ERDDAP returns units in row 2 — skip it
    platforms_df = pd.read_csv(io.StringIO(platforms_resp.text), sep=',', sk
    print(f"Available stations ({len(platforms_df)}):")
    print(platforms_df.to_string(index=False))
except Exception as e:
    print(f"Could not retrieve station list: {e}")
    print("Tip: check the variable names above and adjust the query column a
```

## Step 4 — Query: Temperature and salinity profiles by station and time

In [ ]:
```
# Query parameters — adjust to your dataset and research question
station_id = "vws21g"
variables  = "time,latitude,longitude,depth,TEMP"

# --- Automatically detect the actual time range of the dataset ---
# This avoids hardcoding dates that may fall outside the dataset's coverage
try:
    meta_url = f"{ERDDAP_BASE_URL}/info/{DATASET_ID}/index.csv"
    meta_resp = requests.get(meta_url, timeout=30)
    meta_resp.raise_for_status()
    meta_df = pd.read_csv(io.StringIO(meta_resp.text))

    # Extract actual_range for the time variable
    time_row = meta_df[
        (meta_df['Variable Name'] == 'time') &
        (meta_df['Attribute Name'] == 'actual_range')
    ]
    if not time_row.empty:
        time_range = time_row['Value'].values[0]
        time_start, time_end = [t.strip() for t in time_range.split(',')]
        print(f"Dataset time range: {time_start} → {time_end}")
    else:
        # Fallback: use a broad range and let ERDDAP clip it
        time_start = "2000-01-01T00:00:00Z"
        time_end   = "2100-01-01T00:00:00Z"
        print("Could not detect time range automatically — using full range
except Exception as e:
    time_start = "2000-01-01T00:00:00Z"
    time_end   = "2100-01-01T00:00:00Z"
    print(f"Time range detection failed ({e}) — using full range fallback.")

# --- Build and run the query ---
query_url = (
    f"{dataset_url}.csv"
```

```
    f"?{variables}"
    f'&station_id="{station_id}"'
    f"&time>={time_start}"
    f"&time<={time_end}"
    f'&orderBy("time,depth")'
)

try:
    data_resp = requests.get(query_url, timeout=60)
    data_resp.raise_for_status()
    # ERDDAP returns column names in row 1 and units in row 2 — skip the uni
    data_df = pd.read_csv(io.StringIO(data_resp.text), sep=',', skiprows=[1]
    if data_df.empty:
        print("No data returned. Check the station ID or variable names.")
    else:
        data_df['time'] = pd.to_datetime(data_df['time'])
        print(f"Retrieved {len(data_df)} records for station '{station_id}'"
        print(f"Time range: {data_df['time'].min()} to {data_df['time'].max(
        print(f"Depth range: {data_df['depth'].min():.1f} m to {data_df['dep
        display(data_df.head())
except Exception as e:
    print(f"Query failed: {e}")
    print(f"URL attempted: {query_url}")
    print("Tip: open the URL in a browser to see the ERDDAP error message.")
```

## Step 5 — Visualize: Time series and depth profile

In [ ]:

```
# Guard: only plot if data was retrieved successfully
if 'data_df' not in dir() or data_df.empty:
    print("No data available to plot. Run the query cell first.")
else:
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # --- Left: Temperature time series (surface only, depth <= 10 m) ---
    surface = data_df[data_df['depth'] <= 10].copy()
    if surface.empty:
        axes[0].text(0.5, 0.5, 'No surface data (depth ≤ 10 m)', ha='center'
    else:
        axes[0].plot(surface['time'], surface['TEMP'], color='steelblue', li
    axes[0].set_xlabel("Time")
    axes[0].set_ylabel("Temperature (°C)")
    axes[0].set_title(f"Surface temperature — station {station_id}")
    axes[0].tick_params(axis='x', rotation=30)
    axes[0].grid(True, alpha=0.3)

    # --- Right: Mean temperature profile with depth ---
    profile = data_df.groupby('depth')['TEMP'].mean().reset_index()
    axes[1].plot(profile['TEMP'], profile['depth'], color='darkorange', line
    axes[1].invert_yaxis()  # depth increases downward
    axes[1].set_xlabel("Mean Temperature (°C)")
    axes[1].set_ylabel("Depth (m)")
    axes[1].set_title(f"Mean temperature profile — station {station_id}")
    axes[1].grid(True, alpha=0.3)

    plt.suptitle(f"POLARIN ERDDAP | Dataset: {DATASET_ID}", fontsize=9, colo
```

```
    plt.tight_layout()
    plt.show()
```

### Step 6 — Export data for further analysis

In [ ]:
```
# Save to CSV for use in other tools (MATLAB, R, Excel, etc.)
if 'data_df' not in dir() or data_df.empty:
    print("No data to export. Run the query cell first.")
else:
    output_filename = f"polarin_{DATASET_ID}_{station_id}.csv"
    data_df.to_csv(output_filename, index=False)
    print(f"Data saved to: {output_filename}")
    print(f"Shape: {data_df.shape[0]} rows × {data_df.shape[1]} columns")
```

> **ERDDAP also supports direct download in other formats** — replace `.csv` in the query URL with `.nc` (NetCDF), `.json`, `.mat` (MATLAB), or `.htmlTable` for a quick browser preview. Example:
>
> `https://erddap.s4polarin.eu/erddap/tabledap/{datasetID}.nc?{variables}&{constraints}`

---

# Use Case 2: Student research — Where is there insufficient data?

**Persona**: PhD student or graduate researcher — an intermediate user exploring the polar data landscape.

**Goal**: Investigate data availability across Arctic or Antarctic regions to identify spatial and thematic gaps, and propose areas where additional observations would be valuable.

**Workflow overview**:

1. Review Data Coverage section on the Data Hub landing page.
2. Explore the Data Selection filtered by Location and Earth System Sphere.
3. Identify underrepresented regions.
4. Document gaps and hypothesize where new data collection would be most impactful.

### Step 1 — Check the Data Coverage on the Landing Page

Start at the POLARIN Data Hub landing page.

The Data Coverage panel shows a general dataset overview, including:

- **Total number of datasets** currently available in the Hub.
- **Total number of parameters** covered across all datasets, also divided by ice, marine and atmosphere related parameters.
- The **number of Research Infrastructures (RI)** currently available in the Hub.

These numbers give you a quick sense of the overall data volume. Importantly, they are updated as new datasets are ingested — so they reflect the current state of the Hub.

The second panel, the **Dataset by component in the Earth system** panel lets you filter by:

- **Earth System** (Ocean, Sea Ice, Atmosphere, Glaciers, Permafrost, etc.)

The last panel, the **Dataset by type of research infrastructure** panel lets you filter by:

- **RI** (Arctic RI, Antarctic RI, Vessel, Observatories)

Try clicking on different Earth Systems and note how the dataset count change — this already gives you a spatial and thematic picture of data availability.

> **Exercise**: Record the number of datasets available for each Earth Systems. Which system have the most data? Which have the least?

## Step 2 — Explore the Data Selection

Navigate to the Data Selection and use the filters systematically to assess coverage:

**By Earth Systems**:

- Select one system at a time (e.g., *Sea Ice*) and note how many datasets appear.
- Compare this to a high-coverage system (e.g., *Ocean*).

**By RI**:

- Click on the **RI** filter and explore which Earth Systems are most represented (e.g., Ocean, Atmosphere) and which are rarer (e.g., Permafrost, Lake and rivers).

**By RI type**:

- Use the **RI type** filter and explore where the data are coming from.

**By region**:

- Use the **Location** filter or the station map to compare Arctic vs. Antarctic coverage. Are there geographic areas with no data points?

## Step 3 — Quantify data availability via ERDDAP

For a more quantitative assessment, you can query the ERDDAP to count datasets and their temporal coverage programmatically.

In [ ]:
```
import requests
import pandas as pd
import matplotlib.pyplot as plt
import io
import warnings
warnings.filterwarnings("ignore")

ERDDAP_BASE_URL = "https://erddap.s4polarin.eu/erddap"

# Get full list of available tabledap datasets
# We request datasetID and tabledap URL from the allDatasets virtual table
datasets_url = f"{ERDDAP_BASE_URL}/tabledap/allDatasets.csv?datasetID%2Ctabl

try:
    datasets_resp = requests.get(datasets_url, timeout=30)
    datasets_resp.raise_for_status()
    # ERDDAP returns units in row 2 — skip it
    datasets_df = pd.read_csv(io.StringIO(datasets_resp.text), sep=',', skip
    # Keep only rows that have a tabledap URL (excludes the allDatasets entr
    datasets_df = datasets_df[datasets_df['tabledap'].notna() & (datasets_df
    datasets_df = datasets_df.reset_index(drop=True)
    print(f"Total tabledap datasets available: {len(datasets_df)}")
    display(datasets_df.head(10))
except Exception as e:
    print(f"Could not retrieve dataset list: {e}")
```

In [ ]:
```
# Explore dataset IDs by prefix — POLARIN datasets follow naming conventions
# that often encode the source RI (e.g., 'cnr_iadc_' = CNR/IADC, 'ARICE_' =

if 'datasets_df' not in dir() or datasets_df.empty:
    print("Dataset list not available. Run the previous cell first.")
else:
    datasets_df['prefix'] = datasets_df['datasetID'].str.split('_').str[:2].
    prefix_counts = datasets_df['prefix'].value_counts().head(15)

    fig, ax = plt.subplots(figsize=(10, 5))
    prefix_counts.plot(kind='barh', ax=ax, color='steelblue')
    ax.set_xlabel("Number of datasets")
    ax.set_title("Dataset count by source prefix (top 15)")
    ax.invert_yaxis()
    plt.tight_layout()
    plt.show()
```

```
    print("\nTop 15 source prefixes:")
    print(prefix_counts.to_string())
```

### Step 4 — Identify and document gaps

Based on your exploration in Steps 1–3, document your findings using the structure below. This can serve as the starting point for a research proposal or a data gap report.

**Gap analysis template** (fill in your findings):

| Dimension | Well-covered areas | Data gaps identified |
|---|---|---|
| Earth System | e.g., Ocean | e.g., Permafrost |
| RI | e.g., RV POLARSTERN | e.g., Mario Zucchelli Station |
| Region | e.g., Antarctic | e.g., Arctic |

---

## Use Case 3: Exploring Arctic ocean data for policy purposes

**Persona**: Policy analyst or decision maker — an user who needs clear, actionable insights from polar data without programming.

**Goal**: Get an overview of what ocean observation data is available for the Arctic, explore it visually on the map, and extract plots and underlying data for use in reports or briefings.

**Workflow overview**:

1. Filter the Data Selection by Location and Earth System to find relevant datasets.
2. Browse the results and select datasets of interest.
3. Open the Data Viewer and distinguish data layers from metadata-only entries.
4. Click on data layers (green markers) to explore interactive plots.

### Step 1 — Filter the Data Selection by Location and Earth System

1. Go to the POLARIN Data Selection.
2. Use the **Location** filter and select **Arctic**.

3. Use the **Earth System** filter and select **Ocean** (or another sphere relevant to your topic, e.g., *Sea Ice*, *Atmosphere*).
4. Optionally, filter by **RI** to focus on a specific research infrastructure, or by **RI Type** to narrow down to a particular type of facility (e.g., research station, vessel).

You will see a list of datasets matching your filters. Each entry shows:

- Dataset title and description
- Distributing RI

Moreover, each dataset has a colored markers in the entry. There are two types of markers — and the distinction matters:

| Marker color | What it means |
|---|---|
| **Green** | Data layer/Layers with data — the dataset contains plottable data. Their values can be displayed and plotted in the Data Viewer |

| **Blue** | Metadata only — the dataset has a metadata record in the catalogue, but no plottable data is directly accessible through the Viewer.

> At this stage you may not know exactly what variables each dataset contains — that is normal. The Data Viewer (next step) will help you understand what data is actually plottable and what is available only as metadata.

## Step 2 — Open Selected Datasets in the Data Viewer

To see the same datasets on the Data Viewer, you can apply the same filters as the Data Selection.

Focus your attention on the **green markers** — these are the stations where you can actually explore the data interactively.

## Step 3 — Explore the Data Interactively

Click on any **green marker** on the map. A popup will appear showing:

- The dataset name and the coordinates.
- Relevant links associated with the dataset.
- A link to a **dedicated plot page** — click it to open an interactive time series chart for that station.

In the interactive plot:

- Display the last 30 days of a measurement.
- Hover over the line to read individual data values at specific dates.
- Observe the overall shape of the record — are there seasonal cycles? Long-term trends? Anomalous periods?

Repeat for different green markers to compare stations across the Arctic.

> **For a broader regional picture**: complement station-level exploration with the Reanalysis & Plots dashboard, which provides pre-built visualizations of reanalysis fields (e.g., sea surface temperature, sea ice extent) over the Arctic and Antarctic — no interaction with individual stations needed.

---

# Summary

This notebook walked through three representative use cases for polar data discovery using the POLARIN Data Hub:

**Use Case 1**: An oceanographer used the Data Selection to identify ERDDAP-accessible ocean datasets, retrieved multi-depth temperature data via ERDDAP using Python, produced water column profiles and time series plots, and exported the data for further analysis.

**Use Case 2**: A PhD student explored data availability by Earth System and Location in the Data Selection, identified thematic and geographic gaps, and quantified dataset distribution using the ERDDAP programmatic interface.

**Use Case 3**: A policy analyst filtered the Data Selection by Location (Arctic) and Earth System (Ocean), opened datasets in the Data Viewer, focused on green markers (plottable data layers), and explored interactive station plots to extract insights for policy briefings.

---

## Key takeaways

- The Data Selection is your starting point for structured, filtered data discovery.
- The Data Viewer lets you explore data spatially before downloading — focus on  green markers for plottable data.
- ERDDAP enables precise, reproducible, programmatic access for advanced users.
- All POLARIN data used in publications must be properly cited — include the dataset DOI and the POLARIN grant acknowledgement.

## Further reading

- POLARIN Data Cookbook — for more worked examples of data analysis.
- POLARIN Zenodo Community — for published, citable POLARIN datasets.
- POLARIN PROMPT — for AI-assisted data discovery.
- Reanalysis & Plots — for pre-built dashboards.

In [ ]:

## Find additional data stewardship training resources in the POLARIN Training Resources Database!

POLARIN has prepared a comprehensive list of training resources on data stewardship, covering the entire data lifetime cycle: from data collection, to transformation, curation, analysis, publication, and more. You can find the POLARIN Training Resources Database here. Feel free to contribute to the database by adding your own resources, or by suggesting new resources to be added.

> NOTE: Be sure to navigate to the second sheet to view the data stewardship resources!

## License